

CS 300 Project One: Pseudocode and Runtime Analysis

This document combines the pseudocode for the vector, hash table, and binary search tree approaches for ABCU course advising. The program must load course data, print an alphanumeric course list, print an individual course with prerequisites, and exit through a menu.

Course Object

STRUCT Course: courseNumber, courseTitle, prerequisites list. Each input line is comma-separated as courseNumber, courseTitle, prerequisite1, prerequisite2, etc. A valid line must include at least a course number and title.

Shared Menu Pseudocode

Main Menu

START PROGRAM

DISPLAY menu until user chooses 9

1. Load Courses
2. Print Course List
3. Print Course Information
9. Exit

IF choice = 1: prompt for file name and load file into selected data structure

IF choice = 2: print all courses in alphanumeric order

IF choice = 3: prompt for course number, then print course title and prerequisites

IF choice = 9: exit program

END PROGRAM

Vector Pseudocode

FUNCTION loadVector(fileName)

 OPEN file; IF file cannot open PRINT error and RETURN empty vector

 FOR each line in file

 SPLIT line by commas; IF tokens < 2 PRINT format error and CONTINUE

 CREATE Course using token[0] as courseNumber and token[1] as title

 ADD remaining tokens to Course.prerequisites

 ADD Course to vector

 FOR each Course in vector, verify each prerequisite exists as a courseNumber

 RETURN vector

END FUNCTION

FUNCTION printVectorList(courses)

 SORT courses by courseNumber from lowest to highest

 FOR each Course in courses PRINT courseNumber and courseTitle

END FUNCTION

FUNCTION searchCourse(Vector courses, courseNumber)

 FOR each Course in courses

 IF Course.courseNumber equals courseNumber

 PRINT courseNumber and courseTitle

 FOR each prerequisite PRINT prerequisite course information

 RETURN

 PRINT "Course not found"

END FUNCTION

Hash Table Pseudocode

FUNCTION loadHashTable(fileName)

OPEN file; IF file cannot open PRINT error and RETURN empty hash table

FOR each line in file

SPLIT line by commas; IF tokens < 2 PRINT format error and CONTINUE

CREATE Course with courseNumber, title, and prerequisite list

INSERT Course into hash table using courseNumber as key

STORE courseNumber in validation list

FOR each Course, verify each prerequisite exists in validation list

RETURN hash table

END FUNCTION

FUNCTION printHashTableList(courses)

CREATE temporary list

FOR each bucket in hash table ADD all stored courses to temporary list

SORT temporary list by courseNumber

FOR each Course in temporary list PRINT courseNumber and courseTitle

END FUNCTION

FUNCTION searchCourse(HashTable courses, courseNumber)

USE hash function to locate courseNumber bucket

IF Course is found PRINT courseNumber, title, and prerequisite course information

ELSE PRINT "Course not found"

END FUNCTION

Binary Search Tree Pseudocode

FUNCTION loadTree(fileName)

OPEN file; IF file cannot open PRINT error and RETURN empty tree

FOR each line in file

SPLIT line by commas; IF tokens < 2 PRINT format error and CONTINUE

CREATE Course with courseNumber, title, and prerequisite list

INSERT Course into binary search tree by courseNumber

STORE courseNumber in validation list

FOR each Course, verify each prerequisite exists in validation list

RETURN tree

END FUNCTION

FUNCTION insertNode(currentNode, Course)

IF tree is empty SET Course as root

ELSE IF Course.courseNumber < currentNode.courseNumber move left or insert left

ELSE move right or insert right

END FUNCTION

FUNCTION printTreeList(tree)

PERFORM in-order traversal: left, current, right

PRINT each courseNumber and courseTitle

END FUNCTION

FUNCTION searchCourse(Tree courses, courseNumber)

SET currentNode to root

WHILE currentNode is not null

IF courseNumber equals currentNode.courseNumber PRINT course and prerequisites, then RETURN

ELSE IF courseNumber < currentNode.courseNumber move left

ELSE move right

PRINT "Course not found"

END FUNCTION

Runtime and Memory Analysis

The table below compares the worst-case runtime for reading the file and creating n course objects, plus the main operations required by the advisors. Each stored course requires memory for the course object and prerequisite list, so all three structures use $O(n)$ memory overall.

Data Structure	Load/Create Objects	Print Sorted Course List	Search Course	Memory	Notes
Vector	$O(n)$	$O(n \log n)$ if sorted after load; $O(n)$ if already sorted	$O(n)$	$O(n)$	Simple and easy to implement, but searching is slower because it may scan every course.
Hash Table	$O(n)$ average	$O(n \log n)$ because courses must be gathered and sorted	$O(1)$ average, $O(n)$ worst case	$O(n)$	Fast lookup by courseNumber, but sorted output requires extra sorting. Collisions can reduce

					performance.
Binary Search Tree	$O(n \log n)$ average, $O(n^2)$ worst case if unbalanced	$O(n)$ using in-order traversal	$O(\log n)$ average, $O(n)$ worst case	$O(n)$	Naturally supports sorted output with in-order traversal and provides efficient searching when balanced.

Evaluation and Recommendation

The vector is the simplest structure and works well for a small number of courses, but it is less efficient for searching because the program may need to check every course. The hash table is strong for fast course lookups because it can access records by key, but it does not naturally keep courses in alphanumeric order, so the program needs an extra list and sort step to print the full course list. The binary search tree is useful because its in-order traversal prints courses in alphanumeric order naturally, and searching is efficient when the tree is reasonably balanced.

I recommend using the binary search tree for ABCU's advising program. The advisors need both a sorted course list and the ability to print individual course information. A tree supports sorted output with in-order traversal and provides efficient searching on average. Although an unbalanced tree can become slower in the worst case, the binary search tree best matches the program requirements because it handles both searching and alphanumeric printing without requiring a separate sorting step each time the course list is displayed.